

PRACTICAL_12

1. Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization Correctness and system throughput under different buffer sizes.

CODE:

```
GNU nano 7.2 producer
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    int item;

    for(item = 1; item <= 10; item++)
    {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Produced: %d\n", item);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);

        sleep(1);
    }

    return NULL;
}

void *consumer(void *arg)
```

```

void *consumer(void *arg)
{
    int item;

    for(int i = 1; i <= 10; i++)
    {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumed: %d\n", item);

        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);

        sleep(1);
    }

    return NULL;
}

int main()
{
    pthread_t p, c;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);
}

```

RESULT:

```

root@SRIVALLI:~# nano producer_consumer.c
root@SRIVALLI:~# gcc producer_consumer.c -o producer_consumer -lpthread
root@SRIVALLI:~# ./producer_consumer
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
Produced: 6
Consumed: 6
Produced: 7
Consumed: 7
Produced: 8
Consumed: 8
Produced: 9
Consumed: 9
Produced: 10
Consumed: 10

```

REPORT:

In this experiment, the Producer-Consumer problem was implemented using POSIX threads and counting semaphores. The producer thread generated items and placed them in a shared buffer, while the consumer thread removed and processed them. Semaphores and mutex locks ensured proper synchronization and prevented race conditions. The experiment demonstrates safe communication between threads and efficient resource sharing in concurrent systems.

2. Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

CODE:

```
GNU nano 7.2 deadlock.c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 1 locked Resource 1\n");

    sleep(1);

    pthread_mutex_lock(&resource2);
    printf("Thread 1 locked Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 2 locked Resource 1\n");

    sleep(1);

    pthread_mutex_lock(&resource2);
    printf("Thread 2 locked Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}
```

```

void *thread2(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 2 locked Resource 1\n");

    sleep(1);

    pthread_mutex_lock(&resource2);
    printf("Thread 2 locked Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    return 0;
}

```

RESULT:

```

root@SRIVALLI:~# nano deadlock.c
root@SRIVALLI:~# gcc deadlock.c -o deadlock -lpthread
root@SRIVALLI:~# ./deadlock
Thread 2 locked Resource 1
Thread 2 locked Resource 2
Thread 1 locked Resource 1
Thread 1 locked Resource 2

```

REPORT:

In this experiment, multiple threads competed for shared resources using mutex locks. The deadlock conditions were studied, and a prevention strategy based on resource ordering was implemented. By acquiring resources in the same order, circular waiting was avoided and deadlock was prevented. This experiment helps in understanding deadlock occurrence and prevention in multithreaded systems.